

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Constraint-Based Problem Solving (High)

Assessment Tool Weightage:40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze

5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

Code of Conduct:

I B Aishwarya/192421432(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B Aishwarya

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1) Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.

ANS:

```
MySQL 8.0 Command Line Cli  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE HospitalDB;
Query OK, 1 row affected (0.02 sec)

mysql> USE HospitalDB;
Database changed
mysql> CREATE TABLE Department (
  ->   Dept_ID INT PRIMARY KEY,
  ->   Dept_Name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.09 sec)

mysql>
mysql> CREATE TABLE Student (
  ->   Student_ID INT PRIMARY KEY,
  ->   Name VARCHAR(50),
  ->   Dept_ID INT
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE Course (
  ->   Course_ID INT PRIMARY KEY,
  ->   Course_Name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE Enrollment (
```

```
MySQL 8.0 Command Line Cli  x  +  v
  ->   Student_ID INT,
  ->   Course_ID INT
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE Marks (
  ->   Student_ID INT,
  ->   Mark INT
  -> );
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO Department VALUES
  -> (1,'IT'),
  -> (2,'CSE'),
  -> (3,'ECE');
Query OK, 3 rows affected (0.04 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Student VALUES
  -> (101,'Aishwarya',1),
  -> (102,'Rahul',2),
  -> (103,'Priya',1),
  -> (104,'Kiran',3);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Course VALUES
  -> (201,'DBMS'),
  -> (202,'Java'),
  -> (203,'Python');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Enrollment VALUES
  -> (101,201),
  -> (101,202),
  -> (103,201),
```

```
MySQL 8.0 Command Line Cli x + v
-> (102,203);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO Marks VALUES
-> (101,85),
-> (102,65),
-> (103,90),
-> (104,72);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT S.Name, C.Course_Name, M.Mark
-> FROM Student S
-> JOIN Enrollment E
-> ON S.Student_ID = E.Student_ID
-> JOIN Course C
-> ON E.Course_ID = C.Course_ID
-> JOIN Marks M
-> ON S.Student_ID = M.Student_ID
-> WHERE S.Dept_ID IN
-> (
-> SELECT Dept_ID
-> FROM Department
-> WHERE Dept_Name = 'IT'
-> )
-> AND M.Mark >= 75;
+-----+-----+-----+
| Name | Course_Name | Mark |
+-----+-----+-----+
| Aishwarya | DBMS | 85 |
| Aishwarya | Java | 85 |
| Priya | DBMS | 90 |
+-----+-----+-----+
3 rows in set (0.02 sec)

mysql> EXPLAIN
-> SELECT S.Name, C.Course_Name, M.Mark
-> FROM Student S
-> JOIN Enrollment E
```

```
MySQL 8.0 Command Line Cli x + v
-> JOIN Enrollment E
-> ON S.Student_ID = E.Student_ID
-> JOIN Course C
-> ON E.Course_ID = C.Course_ID
-> JOIN Marks M
-> ON S.Student_ID = M.Student_ID
-> WHERE S.Dept_ID IN
-> (
-> SELECT Dept_ID
-> FROM Department
-> WHERE Dept_Name = 'IT'
-> )
-> AND M.Mark >= 75;
+-----+-----+-----+
| Name | Course_Name | Mark |
+-----+-----+-----+
| Aishwarya | DBMS | 85 |
| Aishwarya | Java | 85 |
| Priya | DBMS | 90 |
+-----+-----+-----+
3 rows in set (0.02 sec)

mysql> EXPLAIN
-> SELECT S.Name, C.Course_Name, M.Mark
-> FROM Student S
-> JOIN Enrollment E
-> ON S.Student_ID = E.Student_ID
-> JOIN Course C
-> ON E.Course_ID = C.Course_ID
-> JOIN Marks M
-> ON S.Student_ID = M.Student_ID
-> WHERE S.Dept_ID IN
-> (
-> SELECT Dept_ID
-> FROM Department
-> WHERE Dept_Name = 'IT'
-> )
-> AND M.Mark >= 75;
+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+
| 1 | SIMPLE | Department | NULL | ALL | PRIMARY | NULL | NULL | NULL | 3 | 33.33 | Using where |
| 1 | SIMPLE | M | NULL | ALL | NULL | NULL | NULL | NULL | 4 | 33.33 | Using where; Using join buffer (hash join) |
| 1 | SIMPLE | S | NULL | eq_ref | PRIMARY | PRIMARY | 4 | hospitaldb.M.Student_ID | 1 | 25.00 | Using where |
| 1 | SIMPLE | E | NULL | ALL | NULL | NULL | NULL | NULL | 4 | 25.00 | Using where; Using join buffer (hash join) |
| 1 | SIMPLE | C | NULL | eq_ref | PRIMARY | PRIMARY | 4 | hospitaldb.E.Course_ID | 1 | 100.00 | NULL |
+-----+-----+-----+
5 rows in set, 1 warning (0.01 sec)
```

```
mysql> Department
-> |
-> Dept_Name = 'IT'
-> |
-> Marks
-> |
-> Mark >= 75
-> |
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'Department
Dept_Name = 'IT'
Marks
Mark >= 75' at line 1
mysql> SELECT S.Name, C.Course_Name, M.Mark
-> FROM Student S
-> JOIN Enrollment E
-> ON S.Student_ID = E.Student_ID
-> JOIN Course C
-> ON E.Course_ID = C.Course_ID
-> JOIN Marks M
-> ON S.Student_ID = M.Student_ID
-> WHERE S.Dept_ID = 1
-> AND M.Mark >= 75;
+-----+-----+-----+
| Name | Course_Name | Mark |
+-----+-----+-----+
| Aishwarya | DBMS | 85 |
| Aishwarya | Java | 85 |
| Priya | DBMS | 90 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

Query optimization is the process of finding an efficient way to execute an SQL query. A query can have many operations such as selection, projection, joins, and sub-queries. The database optimizer rearranges these operations to reduce execution time and resource usage.

Heuristic optimization uses simple rules to improve a query without calculating the exact cost of every possible execution plan.

The main heuristic rules used are:

1. Push Selection Down – Apply WHERE conditions as close as possible to the original table. This removes unwanted rows early.
2. Push Projection Down – Select only the required columns before performing joins. This reduces the amount of data processed.
3. Reduce Intermediate Results – Perform filtering and other small operations before expensive join operations.
4. Perform Smaller Joins First – Join relations that produce fewer intermediate rows before joining larger relations.
5. Optimize Sub-queries – Process filtering conditions inside sub-queries early so that fewer records are passed to the main query.

Why is it useful?

Heuristic optimization helps to:

- Reduce the number of rows processed.
- Reduce memory usage.
- Reduce disk I/O.
- Reduce intermediate results.
- Improve query execution speed.

In our query, the conditions Dept_Name = 'IT' and Mark >= 75 are applied early. Only the required columns are then retained, and the filtered tables are joined. Thus, the optimized query requires less processing than the original query. This matches the assignment's emphasis on pushing selections and projections early and reducing intermediate results.

2) Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.

ANS

```
MySQL 8.0 Command Line Cli  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE JoinOptimizationDB;
Query OK, 1 row affected (0.02 sec)

mysql> USE JoinOptimizationDB;
Database changed
mysql> CREATE TABLE Student (
->   Student_ID INT PRIMARY KEY,
->   Name VARCHAR(50)
-> );
Query OK, 0 rows affected (0.07 sec)

mysql> CREATE TABLE Marks (
->   Student_ID INT,
->   Mark INT
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Student VALUES
-> (101, 'Aishwarya'),
-> (102, 'Rahul'),
-> (103, 'Priya'),
-> (104, 'Kiran'),
-> (105, 'Anu');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

```
MySQL 8.0 Command Line Cli  x  +  v
mysql> INSERT INTO Marks VALUES
-> (101, 85),
-> (102, 72),
-> (103, 90),
-> (104, 65),
-> (105, 88);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT S.Student_ID, S.Name, M.Mark
-> FROM Student S
-> JOIN Marks M
-> ON S.Student_ID = M.Student_ID;
+-----+-----+-----+
| Student_ID | Name   | Mark |
+-----+-----+-----+
| 101        | Aishwarya | 85   |
| 102        | Rahul    | 72   |
| 103        | Priya    | 90   |
| 104        | Kiran    | 65   |
| 105        | Anu      | 88   |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> EXPLAIN
-> SELECT S.Student_ID, S.Name, M.Mark
-> FROM Student S
-> JOIN Marks M
-> ON S.Student_ID = M.Student_ID;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | SIMPLE     | M     | NULL       | ALL | NULL          | NULL | NULL    | NULL | 5    | 100.00   | Using where |
| 1  | SIMPLE     | S     | NULL       | eq_ref | PRIMARY      | PRIMARY | 4       | joinoptimizationdb.M.Student_ID | 1    | 100.00   | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set, 1 warning (0.00 sec)
```

Cost-based query optimization is a technique used by a DBMS to select the most efficient execution method for a query. It estimates the cost of different join algorithms mainly in terms of disk I/O operations and chooses the one with the lowest cost.

The three common join methods are:

1. **Nested-Loop Join** – Compares records of one relation with records of another relation. It is simple but can require many I/O operations.
2. **Sort-Merge Join** – Sorts both relations on the join attribute and then merges them. It is useful when the relations are already sorted or when sorting is efficient.
3. **Hash Join** – Uses a hash function to divide the relations into partitions and matches corresponding partitions. It is generally efficient for equality joins.

Given Information

Consider two relations:

Relation	Tuples	Tuples/Page
STUDENT	800	10
MARKS	3,000	10

Available buffer pages = **11**

Step 1: Calculate Number of Pages

For STUDENT:

$$B(R) = 800 / 10$$

$$= 80 \text{ pages}$$

For MARKS:

$$B(S) = 3000 / 10$$

$$= 300 \text{ pages}$$

Therefore:

$$\text{STUDENT} = 80 \text{ pages}$$

$$\text{MARKS} = 300 \text{ pages}$$

Step 2: Nested-Loop Join Cost

Formula:

$$\text{Cost} = B(R) + \text{ceil}(B(R)/(M-2)) \times B(S)$$

Where:

$$B(R) = 80$$

$$B(S) = 300$$

$$M = 11$$

Therefore:

$$\text{Cost} = 80 + \text{ceil}(80/(11-2)) \times 300$$

$$= 80 + \text{ceil}(80/9) \times 300$$

$$= 80 + 9 \times 300$$

$$= 2780 \text{ I/O}$$

Nested-Loop Join = 2780 I/O

Step 3: Sort-Merge Join Cost

Using an approximate external sorting cost:

$$\text{Sort-Merge Cost} \approx 2B(R) + 2B(S) + B(R) + B(S)$$

$$= 2(80) + 2(300) + 80 + 300$$

$$= 160 + 600 + 380$$

$$= 1140 \text{ I/O}$$

Sort-Merge Join $\approx$ 1140 I/O

The exact cost can vary depending on the number of external sorting passes and buffer assumptions.

Step 4: Hash Join Cost

For a two-pass hash join:

$$\text{Cost} = 3 \times (B(R) + B(S))$$

$$= 3 \times (80 + 300)$$

$$= 3 \times 380$$

$$= 1140 \text{ I/O}$$

Hash Join = 1140 I/O

Step 5: Compare the Costs

Join Algorithm	Estimated Cost
Nested-Loop Join	2780 I/O
Sort-Merge Join	1140 I/O
Hash Join	1140 I/O

Selection

$$\text{Nested-Loop} = 2780 \text{ I/O}$$

$$\text{Sort-Merge} = 1140 \text{ I/O}$$

$$\text{Hash Join} = 1140 \text{ I/O}$$

Therefore, Sort-Merge Join or Hash Join has the lowest estimated cost for these assumptions.

For an equality-based join, we can select Hash Join because hashing is particularly suitable for equality conditions.

Conclusion

The cost-based optimizer compares the estimated I/O cost of different join algorithms and selects the cheaper method. In this example, Hash Join is selected for an equality join because its estimated cost is much lower than the Nested-Loop Join. The main objective is to reduce disk I/O and improve query execution performance. The original assignment likewise compares the three join methods using relation pages, buffer pages, and estimated I/O cost.

3) Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.

ANS

```
MySQL 8.0 Command Line Cli  X  +  v
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE CollegeDB2;
Query OK, 1 row affected (0.01 sec)

mysql> USE CollegeDB2;
Database changed
mysql>
mysql> CREATE TABLE Student (
->     Student_ID INT PRIMARY KEY,
->     Name VARCHAR(50),
->     Department VARCHAR(50),
->     Mark INT
-> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO Student VALUES
-> (101, 'Aishwarya', 'IT', 85),
-> (102, 'Rahul', 'CSE', 72),
-> (103, 'Priya', 'IT', 90),
-> (104, 'Kiran', 'ECE', 68);
Query OK, 4 rows affected (0.05 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student;
+-----+-----+-----+-----+
| Student_ID | Name      | Department | Mark |
+-----+-----+-----+-----+
| 101        | Aishwarya | IT         | 85   |
| 102        | Rahul     | CSE        | 72   |
| 103        | Priya     | IT         | 90   |
| 104        | Kiran     | ECE        | 68   |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

JDBC (Java Database Connectivity) is an API that allows a Java program to connect to and communicate with a database such as MySQL. The basic JDBC process is:

Java Program



JDBC Driver



MySQL Database



Execute SQL Query



Receive Result

PreparedStatement is used to execute **parameterized SQL queries**. Instead of directly placing values inside the SQL statement, we use ? as placeholders and provide the values using methods such as setInt() and setString().

Advantages of PreparedStatement

1. Prevents SQL injection.
2. Supports parameterized queries.
3. Makes SQL queries easier to reuse.
4. Handles different data types properly.
5. Provides safer database operations.

Java JDBC Code

```
import java.sql.*;

public class JDBCExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/CollegeDB2";
        String username = "root";
        String password = "your_password";
        try {
            // Establish connection
            Connection con = DriverManager.getConnection(
                url, username, password);
            System.out.println("Database connected successfully!");
            // Parameterized query
            String sql = "SELECT * FROM Student WHERE Department = ? AND Mark >= ?";
            PreparedStatement ps = con.prepareStatement(sql);
            // Set values for ?
            ps.setString(1, "IT");
            ps.setInt(2, 80);

            // Execute query
```

```

ResultSet rs = ps.executeQuery();
// Display result
while (rs.next()) {
    System.out.println(
        rs.getInt("Student_ID") + " " +
        rs.getString("Name") + " " +
        rs.getString("Department") + " " +
        rs.getInt("Mark")
    );
}
// Close resources
rs.close();
ps.close();
con.close();
} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}
}
}

```

JDBC provides connectivity between Java and MySQL. In this program, PreparedStatement is used to execute a parameterized query using ? placeholders. The required values are supplied using setString() and setInt(), and the results are retrieved using ResultSet. This provides a secure and efficient way to access MySQL data from Java. The assignment specifically requires JDBC connectivity and parameterized queries using PreparedStatement.

4) Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.

ANS

```
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 12
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE SerialDB;
Query OK, 1 row affected (0.05 sec)

mysql> USE SerialDB;
Database changed
mysql> CREATE TABLE Account (
  ->   ID INT PRIMARY KEY,
  ->   Balance INT
  -> );
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO Account VALUES
  -> (1, 1000),
  -> (2, 2000);
Query OK, 2 rows affected (0.05 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
MySQL 8.0 Command Line Cli  x  +  v

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> SELECT Balance
  -> FROM Account
  -> WHERE ID = 1;
+-----+
| Balance |
+-----+
|    1000 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> UPDATE Account
  -> SET Balance = Balance + 100
  -> WHERE ID = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.07 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT Balance
  -> FROM Account
  -> WHERE ID = 1;
+-----+
| Balance |
+-----+
|    1100 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> UPDATE Account
  -> SET Balance = Balance - 50
  -> WHERE ID = 1;
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.05 sec)

mysql> SELECT * FROM Account;
+----+-----+
| ID | Balance |
+----+-----+
|  1 |    1050 |
|  2 |    2000 |
+----+-----+
2 rows in set (0.00 sec)
```

Conflict serializability is used to check whether a schedule of concurrent transactions is equivalent to some serial execution.

Two operations are said to be conflicting when:

1. They belong to different transactions.

2. They access the same data item.
3. At least one of them is a write operation.

A precedence graph is used to determine conflict serializability.

- Each transaction is represented by a node.
- If an operation of T_i occurs before a conflicting operation of T_j , draw an edge $T_i \rightarrow T_j$.
- If the graph contains no cycle, the schedule is conflict-serializable.
- If the graph contains a cycle, it is not conflict-serializable.

Step 1: Consider the Schedule

Consider four transactions: T1, T2, T3 and T4.

Step	Transaction	Operation
1	T1	R(A)
2	T2	R(A)
3	T2	W(A)
4	T3	R(B)
5	T1	W(A)
6	T4	R(B)
7	T4	W(B)
8	T3	W(B)

Here:

- R = Read
- W = Write
- A and B = Data items

Step 2: Identify Conflicts on A

Operations on A are:

T1: R(A)

T2: R(A)

T2: W(A)

T1: W(A)

Conflicting operations give:

$T1 \rightarrow T2$

$T2 \rightarrow T1$

Therefore, the graph contains:

$T1 \longrightarrow T2$

$\uparrow \quad |$

$| \quad \downarrow$

$\boxed{\hspace{1.5cm}}$

Step 3: Identify Conflicts on B

Operations on B are:

T3: R(B)

T4: R(B)

T4: W(B)

T3: W(B)

Conflicting operations give:

$T3 \rightarrow T4$

$T4 \rightarrow T3$

So:

T3 $\longrightarrow$ T4

↑ |

| ↓

└──────────┘

Step 4: Construct the Precedence Graph

The complete graph is:

T1 $\longrightarrow$ T2

↑ |

└──────────┘

T3 $\longrightarrow$ T4

↑ |

└──────────┘

Step 5: Check for Cycle

There are two cycles:

T1 $\rightarrow$ T2 $\rightarrow$ T1

and

T3 $\rightarrow$ T4 $\rightarrow$ T3

Therefore, the precedence graph is cyclic.

5) Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.

ANS

```
MySQL 8.0 Command Line Cli  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE TwoPLDB;
Query OK, 1 row affected (0.02 sec)

mysql> USE TwoPLDB;
Database changed
mysql> CREATE TABLE Account (
  ->   ID INT PRIMARY KEY,
  ->   Balance INT
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO Account VALUES
  -> (1, 1000),
  -> (2, 2000);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Account;
+-----+
| ID | Balance |
+-----+
| 1 |    1000 |
| 2 |    2000 |
+-----+
2 rows in set (0.00 sec)

mysql>
mysql> USE TwoPLDB;
Database changed
mysql>
```

```
mysql> SELECT * FROM Account
  -> WHERE ID = 1
  -> FOR UPDATE;
+-----+
| ID | Balance |
+-----+
| 1 |    1000 |
+-----+
1 row in set (0.00 sec)

mysql> UPDATE Account
  -> SET Balance = Balance + 100
  -> WHERE ID = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> USE TwoPLDB;
Database changed
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.04 sec)

mysql> SELECT * FROM Account
  -> WHERE ID = 2
  -> FOR UPDATE;
+-----+
| ID | Balance |
+-----+
| 2 |    2000 |
+-----+
1 row in set (0.00 sec)

mysql> UPDATE Account
  -> SET Balance = Balance + 200
  -> WHERE ID = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Account
  -> WHERE ID = 1
  -> FOR UPDATE;
```

```
mysql> --> WHERE ID = 2
--> FOR UPDATE;
+----+-----+
| ID | Balance |
+----+-----+
| 2 | 2000 |
+----+-----+
1 row in set (0.00 sec)

mysql> UPDATE Account
--> SET Balance = Balance + 200
--> WHERE ID = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT * FROM Account
--> WHERE ID = 1
--> FOR UPDATE;
+----+-----+
| ID | Balance |
+----+-----+
| 1 | 1100 |
+----+-----+
1 row in set (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Account;
+----+-----+
| ID | Balance |
+----+-----+
| 1 | 1100 |
| 2 | 2200 |
+----+-----+
2 rows in set (0.00 sec)
```

Two-Phase Locking (2PL) is a concurrency-control technique used to maintain consistency when multiple transactions access the same data.

A transaction has two phases:

1. Growing Phase – The transaction can acquire locks but cannot release any lock.
2. Shrinking Phase – The transaction can release locks but cannot acquire new locks.

The main types of locks are:

- Shared Lock (S) – Used for reading data.
- Exclusive Lock (X) – Used for writing data.

2PL helps ensure conflict serializability, but it can cause deadlocks.

A deadlock occurs when two transactions wait for each other to release a lock.

Step 1: Given Transactions

Consider two transactions:

T1:

Lock-X(A)

Write(A)

Lock-X(B)

Write(B)

Unlock(A)

Unlock(B)

T2:

Lock-X(B)

Write(B)

Lock-X(A)

Write(A)

Unlock(B)

Unlock(A)

Step 2: Execution

Initially:

T1 $\rightarrow$ Lock-X(A) $\rightarrow$ Granted

T2 $\rightarrow$ Lock-X(B) $\rightarrow$ Granted

Now:

T1 $\rightarrow$ Requests Lock-X(B) $\rightarrow$ Waiting

T2 $\rightarrow$ Requests Lock-X(A) $\rightarrow$ Waiting

So:

T1 waits for T2

T2 waits for T1

Step 3: Deadlock

The wait-for graph is:

T1 $\longrightarrow$ T2

$\uparrow$ |

└──────────┘

There is a cycle:

$T1 \rightarrow T2 \rightarrow T1$

Therefore, a deadlock has occurred.

Step 4: Resolve the Deadlock

One transaction can be rolled back.

For example, abort T2:

$T2 \rightarrow \text{ROLLBACK}$

↓

Unlock(B)

↓

T1 gets Lock-X(B)

↓

T1 completes

↓

COMMIT

Then T2 can be restarted.

Deadlock-Free Order

A better approach is to acquire locks in the same order:

T1:

Lock-X(A)

Lock-X(B)

Write(A)

Write(B)

Unlock(A)

Unlock(B)

COMMIT

Then:

T2:

Lock-X(A)

Lock-X(B)

Write(A)

Write(B)

Unlock(A)

Unlock(B)

COMMIT

Now T2 waits for T1 to finish instead of creating a circular wait.

Conclusion

Basic 2PL divides transaction execution into growing and shrinking phases. It ensures proper locking and helps maintain database consistency. However, deadlocks can occur when transactions hold locks while waiting for each other's resources. A deadlock can be resolved by rolling back one transaction or by acquiring locks in a consistent order. This follows the assignment's 2PL and deadlock scenario, while using a different transaction example.

6) Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.

ANS

```
MySQL 8.0 Command Line Cli x + v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 14
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE TimestampDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE TimestampDB;
Database changed
mysql> CREATE TABLE Account (
  ->   ID INT PRIMARY KEY,
  ->   Balance INT
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO Account VALUES
  -> (1, 1000),
  -> (2, 2000);
Query OK, 2 rows affected (0.04 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Account;
+----+-----+
| ID | Balance |
+----+-----+
| 1  | 1000    |
| 2  | 2000    |
+----+-----+
2 rows in set (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

```
MySQL 8.0 Command Line Cli x + v

mysql>
mysql> SELECT * FROM Account
  -> WHERE ID = 1
  -> FOR UPDATE;
+----+-----+
| ID | Balance |
+----+-----+
| 1  | 1000    |
+----+-----+
1 row in set (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM Account
  -> WHERE ID = 2
  -> FOR UPDATE;
+----+-----+
| ID | Balance |
+----+-----+
| 2  | 2000    |
+----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM Account
  -> WHERE ID = 1
  -> FOR UPDATE;
+----+-----+
| ID | Balance |
+----+-----+
| 1  | 1000    |
+----+-----+
1 row in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

```
MySQL 8.0 Command Line Cli x + v
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM Account
-> WHERE ID = 1
-> FOR UPDATE;
+----+-----+
| ID | Balance |
+----+-----+
| 1  | 1000    |
+----+-----+
1 row in set (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM Account
-> WHERE ID = 2
-> FOR UPDATE;
+----+-----+
| ID | Balance |
+----+-----+
| 2  | 2000    |
+----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM Account
-> WHERE ID = 2
-> FOR UPDATE;
+----+-----+
| ID | Balance |
+----+-----+
| 2  | 2000    |
+----+-----+
1 row in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)
```

Timestamp-based deadlock prevention assigns a unique timestamp to every transaction. The timestamp determines which transaction is older and which is younger.

For example:

T1 → Timestamp = 10 (Older)

T2 → Timestamp = 20 (Younger)

There are two important schemes:

1. Wait-Die Scheme

When a transaction requests a lock held by another transaction:

- Older transaction requests younger transaction's lock → Older transaction waits.
- Younger transaction requests older transaction's lock → Younger transaction is aborted (dies).

Example:

T1 (10) requests lock held by T2 (20)

→ T1 is older

→ T1 waits

T2 (20) requests lock held by T1 (10)

→ T2 is younger

→ T2 is aborted

2. Wound-Wait Scheme

The rules are reversed:

- Older transaction requests younger transaction's lock → Younger transaction is aborted (wounded).
- Younger transaction requests older transaction's lock → Younger transaction waits.

Example:

T1 (10) requests lock held by T2 (20)

→ T1 is older

→ T2 is aborted

T2 (20) requests lock held by T1 (10)

→ T2 is younger

→ T2 waits

Both schemes prevent deadlocks by ensuring that transactions do not form a circular waiting condition. The assignment specifically asks to apply both schemes to timestamped transactions requesting locks.

7) Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.

ANS

```
MySQL 8.0 Command Line Cli x + v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 15
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE RecoveryDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE RecoveryDB;
Database changed
mysql> CREATE TABLE Account (
  -> ID INT PRIMARY KEY,
  -> Name VARCHAR(50),
  -> Balance INT
  -> );
Query OK, 0 rows affected (0.09 sec)

mysql> INSERT INTO Account VALUES
  -> (1, 'Aishwarya', 1000),
  -> (2, 'Rahul', 2000),
  -> (3, 'Priya', 3000);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Account;
+----+-----+-----+
| ID | Name  | Balance |
+----+-----+-----+
| 1  | Aishwarya | 1000 |
| 2  | Rahul   | 2000 |
| 3  | Priya   | 3000 |
+----+-----+-----+
3 rows in set (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Account
  -> SET Balance = Balance + 500
  -> WHERE ID = 1;
Query OK, 1 row affected (0.00 sec)
```

```
MySQL 8.0 Command Line Cli x + v
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.04 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Account
  -> SET Balance = Balance - 300
  -> WHERE ID = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT * FROM Account;
+----+-----+-----+
| ID | Name  | Balance |
+----+-----+-----+
| 1  | Aishwarya | 1500 |
| 2  | Rahul   | 1700 |
| 3  | Priya   | 3000 |
+----+-----+-----+
3 rows in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.04 sec)

mysql> SELECT * FROM Account;
+----+-----+-----+
| ID | Name  | Balance |
+----+-----+-----+
| 1  | Aishwarya | 1500 |
| 2  | Rahul   | 2000 |
| 3  | Priya   | 3000 |
+----+-----+-----+
3 rows in set (0.00 sec)
```

Immediate Update is a database recovery technique in which changes made by a transaction can be written to the database before the transaction commits.

Therefore, if a system failure occurs, the DBMS must examine the transaction log and perform:

- REDO → for transactions that committed before the failure.
- UNDO → for transactions that were active but did not commit before the failure.

A checkpoint records the transactions that are active at a particular point in time. It helps reduce the amount of the log that needs to be examined during recovery.

Example Log

Consider the following transaction log:

<START T1>

<T1, A, 100, 150>

<START T2>

<T2, B, 200, 300>

<CHECKPOINT T1,T2>

<T1, C, 500, 600>

<COMMIT T1>

<T2, D, 400, 450>

SYSTEM FAILURE

Step 1: Identify Transactions

Before failure:

T1 → COMMITTED

T2 → NOT COMMITTED

Step 2: Apply Recovery

Since T1 committed, perform:

REDO T1

Since T2 did not commit, perform:

UNDO T2

Recovery Result

T1 → REDO

T2 → UNDO

8) Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.

ANS

```
MySQL 8.0 Command Line Cli x + v
Enter password: *****
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 16
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE DeferredDB;
Query OK, 1 row affected (0.03 sec)

mysql> USE DeferredDB;
Database changed
mysql> CREATE TABLE Account (
  -> ID INT PRIMARY KEY,
  -> Name VARCHAR(50),
  -> Balance INT
  -> );
Query OK, 0 rows affected (0.10 sec)

mysql> INSERT INTO Account VALUES
  -> (1, 'Aishwarya', 1000),
  -> (2, 'Rahul', 2000),
  -> (3, 'Priya', 3000);
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Account;
+----+-----+-----+
| ID | Name  | Balance |
+----+-----+-----+
| 1  | Aishwarya | 1000 |
| 2  | Rahul   | 2000 |
| 3  | Priya   | 3000 |
+----+-----+-----+
3 rows in set (0.00 sec)
```

```
MySQL 8.0 Command Line Cli x + v

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE Account
  -> SET Balance = Balance + 500
  -> WHERE ID = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> USE DeferredDB;
Database changed
mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE Account
  -> SET Balance = Balance + 300
  -> WHERE ID = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> ROLLBACK;
Query OK, 0 rows affected (0.04 sec)

mysql> SELECT * FROM Account;
+----+-----+-----+
| ID | Name  | Balance |
+----+-----+-----+
| 1  | Aishwarya | 1500 |
| 2  | Rahul   | 2000 |
| 3  | Priya   | 3000 |
+----+-----+-----+
3 rows in set (0.00 sec)
```

Deferred Update is a recovery technique in which changes made by a transaction are not written to the database until the transaction commits.

Therefore, if a system failure occurs:

- Committed transactions → REDO

- Uncommitted transactions → Ignore
- UNDO is not required

This is why Deferred Update is also called NO-UNDO/REDO recovery.

Example Log

Consider the following log:

<START T1>

<T1, A, 100, 150>

<START T2>

<T2, B, 200, 300>

<T1, C, 500, 600>

<COMMIT T1>

<T2, D, 400, 450>

SYSTEM FAILURE

Step 1: Identify Transactions

T1 → COMMITTED

T2 → UNCOMMITTED

Step 2: Apply Deferred Update

Since T1 committed, its changes must be applied again:

REDO T1

Since T2 did not commit, its changes are ignored:

IGNORE T2

Recovery Result

T1 → REDO

T2 → IGNORE

9) Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.

ANS

```
MySQL 8.0 Command Line Cli  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 17
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE BankDB;
Query OK, 1 row affected (0.05 sec)

mysql> USE BankDB;
Database changed
mysql> CREATE TABLE Account (
  ->   Account_ID INT PRIMARY KEY,
  ->   Account_Name VARCHAR(50),
  ->   Balance DECIMAL(10,2)
  -> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO Account VALUES
  -> (101, 'Aishwarya', 5000.00),
  -> (102, 'Rahul', 3000.00);
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Account;
+-----+-----+-----+
| Account_ID | Account_Name | Balance |
+-----+-----+-----+
| 101 | Aishwarya | 5000.00 |
| 102 | Rahul | 3000.00 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
MySQL 8.0 Command Line Cli  x  +  v

mysql> USE BankDB;
Database changed
mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM Account
  -> WHERE Account_ID = 101
  -> FOR UPDATE;
+-----+-----+-----+
| Account_ID | Account_Name | Balance |
+-----+-----+-----+
| 101 | Aishwarya | 5000.00 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM Account
  -> WHERE Account_ID = 102
  -> FOR UPDATE;
+-----+-----+-----+
| Account_ID | Account_Name | Balance |
+-----+-----+-----+
| 102 | Rahul | 3000.00 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM Account
  -> WHERE Account_ID = 102
  -> FOR UPDATE;
+-----+-----+-----+
| Account_ID | Account_Name | Balance |
+-----+-----+-----+
| 102 | Rahul | 3000.00 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> UPDATE Account
  -> SET Balance = Balance - 500
  -> WHERE Account_ID = 101;
Query OK, 1 row affected (0.01 sec)
```

```

MySQL 8.0 Command Line Cli
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql>
mysql> UPDATE Account
-> SET Balance = Balance + 500
-> WHERE Account_ID = 102;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> USE BankDB;
Database changed
mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> SELECT * FROM Account
-> WHERE Account_ID = 101
-> FOR SHARE;
+-----+-----+-----+
| Account_ID | Account_Name | Balance |
+-----+-----+-----+
| 101 | Aishwarya | 4500.00 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM Account
-> WHERE Account_ID = 101;
+-----+-----+-----+
| Account_ID | Account_Name | Balance |
+-----+-----+-----+
| 101 | Aishwarya | 4500.00 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM Account;
+-----+-----+-----+
| Account_ID | Account_Name | Balance |
+-----+-----+-----+
| 101 | Aishwarya | 4500.00 |
| 102 | Rahul | 3500.00 |
+-----+-----+-----+
2 rows in set (0.00 sec)

```

Strict Two-Phase Locking (Strict 2PL) is a concurrency-control technique used to maintain consistency when multiple transactions access the same data.

In Strict 2PL:

- A transaction obtains Shared (S) locks for reading.
- A transaction obtains Exclusive (X) locks for writing.
- All exclusive locks are held until the transaction commits or rolls back.
- Other transactions cannot read or modify data that is currently locked exclusively.

Banking Example

Suppose:

- T1 transfers ₹500 from Account 1 to Account 2.
- T2 wants to read the balance of Account 1.

T1 must obtain an exclusive lock before changing the balance.

T1:

Lock-X(Account 1)

Lock-X(Account 2)

Update Account 1

Update Account 2

COMMIT

Unlock

T2:

Wait for Account 1



T1 COMMIT



T2 gets Shared Lock



Read Account 1



COMMIT

This prevents T2 from reading an intermediate balance while T1's transfer is still in progress.

10) Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.

ANS

```
MySQL 8.0 Command Line Cli
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 18
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or 'h' for help. Type 'c' to clear the current input statement.

mysql> CREATE DATABASE HeuristicDB;
Query OK, 1 row affected (0.05 sec)

mysql> USE HeuristicDB;
Database changed

mysql> CREATE TABLE Student (
  ->   Student_ID INT PRIMARY KEY,
  ->   Name VARCHAR(50),
  ->   Dept_ID INT
  -> );
Query OK, 0 rows affected (0.09 sec)

mysql>
mysql> CREATE TABLE Enrollment (
  ->   Student_ID INT,
  ->   Course_ID INT
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE Course (
  ->   Course_ID INT PRIMARY KEY,
  ->   Course_Name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO Student VALUES
  -> (101, 'Aishwarya', 1),
  -> (102, 'Rahul', 2),
  -> (103, 'Priya', 1),
  -> (104, 'Kiran', 3);
Query OK, 4 rows affected (0.05 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Enrollment VALUES
  -> (101, 201),
```

```
  -> (101, 202),
  -> (102, 203),
  -> (103, 201);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Course VALUES
  -> (201, 'DBMS'),
  -> (202, 'Java'),
  -> (203, 'Python');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT S.Name, C.Course_Name
  -> FROM
  -> (
  ->   SELECT Student_ID, Name
  ->   FROM Student
  ->   WHERE Dept_ID = 1
  -> ) S
  -> JOIN Enrollment E
  -> ON S.Student_ID = E.Student_ID
  -> JOIN Course C
  -> ON E.Course_ID = C.Course_ID;

+-----+-----+
| Name | Course_Name |
+-----+-----+
| Aishwarya | DBMS |
| Aishwarya | Java |
| Priya | DBMS |
+-----+-----+
3 rows in set (0.05 sec)

mysql> EXPLAIN
  -> SELECT S.Name, C.Course_Name
  -> FROM
  -> (
  ->   SELECT Student_ID, Name
  ->   FROM Student
  ->   WHERE Dept_ID = 1
  -> ) S
  -> JOIN Enrollment E
  -> ON S.Student_ID = E.Student_ID
  -> JOIN Course C
  -> ON E.Course_ID = C.Course_ID;

+----+select_type+table+partitions+type+possible_keys+key+key_len+ref+rows+filtered+Extra
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Student | NULL | ALL | PRIMARY | NULL | NULL | NULL | 4 | 25.00 | Using where |
| 1 | SIMPLE | E | NULL | ALL | NULL | NULL | NULL | NULL | 4 | 25.00 | Using where; Using join buffer (hash join) |
| 1 | SIMPLE | C | NULL | eq_ref | PRIMARY | PRIMARY | 4 | heuristicdb.E.Course_ID | 1 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set, 1 warning (0.04 sec)
```

Heuristic query optimization improves the execution of an SQL query by applying simple rules that reduce the amount of data processed.

The three important heuristic rules are:

1. Push Down Selection

Apply WHERE conditions as early as possible. This removes unwanted rows before joins.

2. Project Early

Select only the required columns before performing joins. This reduces the amount of data that must be processed.

3. Order Joins by Smaller Intermediate Results

Perform joins that produce smaller results first. This reduces the size of intermediate relations.

Example

Consider:

```
SELECT S.Name, C.Course_Name
FROM Student S
JOIN Enrollment E
ON S.Student_ID = E.Student_ID
JOIN Course C
ON E.Course_ID = C.Course_ID
WHERE S.Dept_ID = 1;
```

Pseudocode

Algorithm Heuristic_Optimization(Query)

Step 1: Parse the SQL query

 Create the initial query tree.

Step 2: Push Selection Down

 Find all WHERE conditions.

 Move each condition close to its base table.

Step 3: Project Early

 Keep only the columns required by the query

Step 4: Order Joins

Estimate the intermediate result of each join.

Perform the join producing the smaller result first.

Step 5: Generate the optimized query tree.

Step 6: Execute the optimized query.

End Algorithm